

Zero-Copy Import v1.2

Removing the in-VRAM copy floor for arbitrary CUDA tensors
in Caliper’s tensor bridge

Caliper project — branch `feat/zerocopy-arbitrary-cuda`

July 7, 2026

Abstract

Caliper’s tensor bridge turns live `torch` tensors into GPU textures across a frozen C ABI. On the Vulkan/CUDA backend, an *arbitrary* framework-allocated CUDA tensor could reach a texture with zero host copies, but never with zero copies outright: memory born in torch’s caching allocator is not exportable to Vulkan, so every texture update paid one in-VRAM `cuMemcpyDtoD` into a host-owned interop buffer. This report describes Bridge v1.2, which removes that floor by moving the question from *copy time* to *allocation time*: an applet-side exportable memory pool routes chosen torch allocations into `cuMemCreate` blocks carrying shareable OS handles; the host imports each block into Vulkan *once* and runs texture updates directly from it at byte offsets. The design is additive (no ABI break), fails closed at every gate (a refused update is a `false` and a fallback, never a crash or a wrong image), and is hardware-verified on an NVIDIA RTX 500 Ada (driver 596.47): byte-exact readback tests at multiple offsets, a live training applet whose status line reports *zero-copy (imported pool)*, and an intact CPU-staged fallback under a GL renderer. The in-VRAM copy floor is now *per allocation origin*: it persists only for memory born unshareable.

1 Background

Caliper hosts small ML “applets” behind a stable C ABI. An applet computes with `libtorch` on whatever device it negotiated; the host renders its UI and owns the GPU. The *tensor bridge* service (`caliper.tensor_bridge.v1`) is the seam between the two: the applet hands over a `CaliperTensor` descriptor, the host materializes or updates a texture, and the applet draws it through `ImGui`. The v1/v1.1 headers and dispatch tables are frozen; revisions are additive (prefix-identical structs, new capability bits), so old applets run on new hosts and vice versa.

Before this work the bridge had a ladder of data-crossing costs, best to worst: `alloc_shared` (the applet’s kernels write texture-backed VRAM in place — literal zero-copy, but the *host* chooses the allocation), the CUDA interop path for arbitrary device tensors (zero host copies, one in-VRAM copy), and CPU staging (the portable worst case). The middle rung is the subject of this report.

2 The problem: memory born unshareable

Vulkan can read external device memory only if that memory was *allocated for export*: on Windows, a `cuMemCreate` allocation with `CU_MEM_HANDLE_TYPE_WIN32` yields an opaque NT handle that `vkAllocateMemory` can import. Memory obtained from plain `cudaMalloc` — which is what torch’s caching allocator uses — carries no such handle and cannot be retrofitted with one. Exportability is decided at *allocation time*, by allocation *flags*.

Consequently, for an arbitrary torch CUDA tensor the host can do no better than: keep a host-owned, exportable interop buffer behind each texture, and on every update copy the

tensor’s bytes into it with `cuMemcpyDtoD` (Figure 1). The copy is device-to-device and cheap in absolute terms, but it is paid *per update, forever*, it doubles the working-set touch for large maps, and it is a standing asterisk on the platform’s zero-copy story: the whitepaper had to state a “one in-VRAM-copy floor” for framework-allocated tensors.

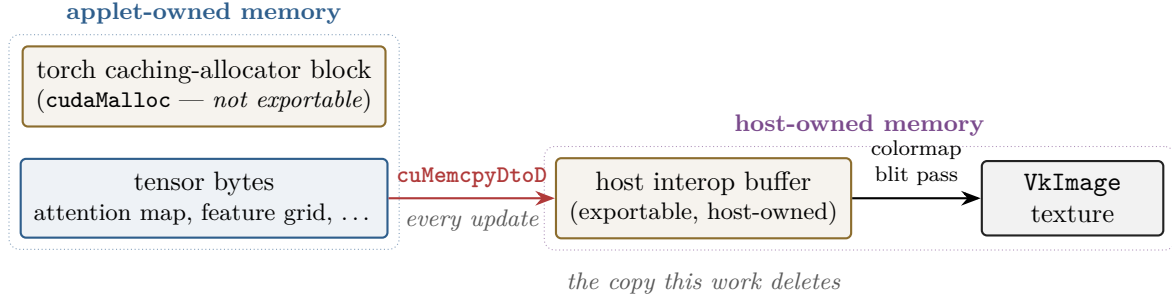


Figure 1: The v1 interop path for an arbitrary CUDA tensor. The tensor’s memory cannot be imported by Vulkan, so its bytes are copied in VRAM into a host-owned exportable buffer on *every* texture update — the “in-VRAM copy floor.”

Two constraints rule out the obvious dodges. The host cannot simply allocate all applet tensors itself (`alloc_shared` exists, but forcing every intermediate through host allocations would invert ownership of the applet’s compute). And the ABI cannot change shape: v1/v1.1 applets must keep working bit-for-bit.

3 The idea: change where memory is born

If exportability is decided at allocation time, then the fix belongs at allocation time. Torch 2.5 provides the two hooks needed to apply it *selectively*, from the applet side, without forking torch:

1. `CUDAPluggableAllocator` — user-supplied alloc and free hooks behind torch’s caching allocator; and
2. `c10::cuda::MemPool + beginAllocateToPool` — a scope that routes the allocations made *inside it* to a chosen pool (the mechanism CUDA graphs use).

The SDK’s new `ExportablePool` combines them: its allocation function builds blocks with `cuMemCreate` (+ WIN32 shareable handle type), maps them with `cuMemMap/cuMemSetAccess`, exports the OS handle, and registers the block in an interval map (`AllocRegistry`: pointer → (block, byte offset)). An applet materializes exactly the tensors it wants to display inside `pool.use()`; everything else in the process keeps torch’s default allocator. The host imports each block *once* and thereafter updates textures straight out of it (Figure 2).

The floor therefore becomes a statement about *origin*, not about frameworks: tensors born in the pool update with zero copies of their data; tensors born unshareable (default torch allocator, any foreign allocator) keep the one-copy interop path, unchanged.

4 Architecture

Figure 3 shows the pieces and who owns them. The ABI grows by one additive revision, `caliper.tensor_bridge.v1_2`: the same seven members as v1.1, prefix-identical, plus three entry points and one capability bit.

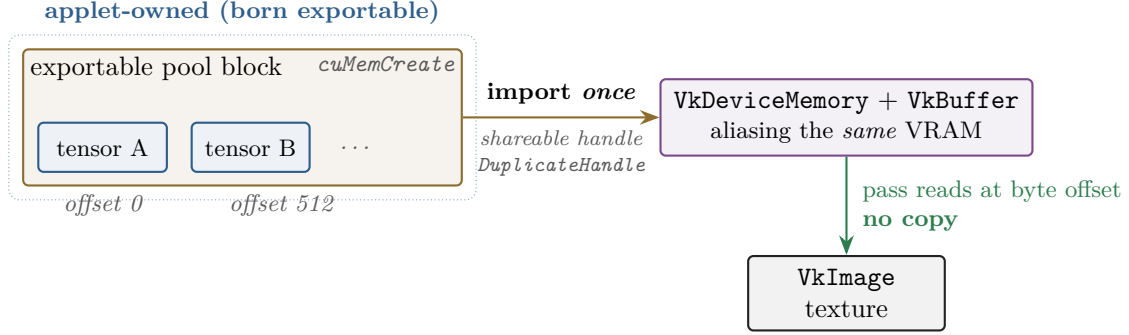


Figure 2: The v1.2 imported path. The pool block is imported into Vulkan once; every tensor inside it is addressed by byte offset, and the colormap/blit pass reads the tensor’s bytes *in place*. The per-update `cuMemcpyDtoD` of Figure 1 no longer exists. (Blocks are 2 MiB-granularity `cuMemCreate` allocations; torch sub-allocates tensors at 512-byte alignment.)

```

/* caps() bit 1: the host can import applet-exported allocations. */
#define CALIPER_BRIDGE_CAP_IMPORT_ALLOC (1u << 1)

CaliperAllocId import_allocation(void* os_handle, uint64_t size_bytes,
                                uint32_t handle_type); /* 0 = declined */
void          release_allocation(CaliperAllocId alloc);
bool          update_texture_from_alloc(CaliperTextureId tex,
                                        CaliperAllocId alloc,
                                        uint64_t offset_bytes,
                                        const CaliperTensor* desc);

```

`desc` carries shape/dtype/strides/stream; its `data` pointer is ignored — the (allocation, offset) pair *is* the address. The header also states the memory-stability contract: the pass reads the imported bytes in place, so the window [offset, offset + extent) must not be rewritten until the texture’s next update.

4.1 The update path

For an f32 mapped texture the colormap compute pass binds the imported `VkBuffer` with a *descriptor offset* equal to the tensor’s byte offset — the shader is unchanged and never knows it is reading foreign memory. For u8 RGBA the blit path sets `bufferOffset` in the buffer-to-image copy region. Both device paths report distinct telemetry ("`compute-imported`" / "`blit-imported`") so tests and status lines can claim exactly the path that ran. Synchronization reuses the existing V4 machinery: the pipelined chain signals the shared timeline semaphore *without* a preceding copy, and where timeline semaphores are unavailable the fenced synchronous fallback applies.

4.2 Failing closed

Every gate refuses by returning `false` or 0, leaving pixels and telemetry untouched; the caller falls through to the copying path. Gates include: capability bit absent; handle import failure (unwound completely); offset misaligned for the f32 descriptor-offset path — the Vulkan `minStorageBufferOffsetAlignment` limit; torch sub-allocations are 512-aligned, so real pool tensors pass; window out of bounds, checked overflow-safely at *both* the bridge and the renderer, phrased as `offset > size || bytes > size - offset` so no sum can wrap; allocation already released (ids are never reused); and a buffer whose memory requirements exceed the imported size (logged, declined). A failed import is never a crash and never a wrong image.

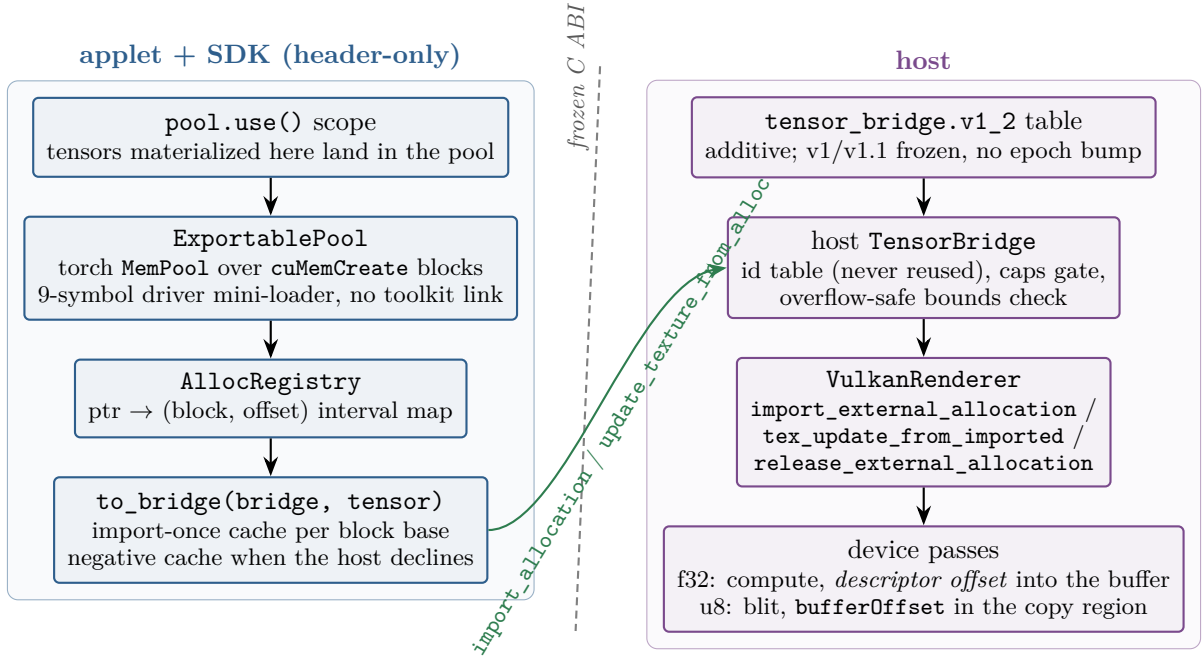


Figure 3: Component view. The SDK side is header-only and loads its nine `cuMem*` driver symbols itself (no CUDA toolkit link, per the platform’s dependency rules); the host side is one additive dispatch table over existing renderer seams. Import happens once per block and is cached by block base address on the applet side.

4.3 Lifetime

Block frees are initiated by torch’s allocator, which may run its callback on any thread; the bridge, by contract, is frame-thread-only. The pool therefore *queues* host-import releases in the free callback and drains the queue at its two frame-thread call sites (`to_bridge` and the destructor). The destructor additionally releases the import of every block it reclaims itself, so correctness does not depend on whether torch’s `releasePool` frees cached segments synchronously. The exemplar applet guards the last edge: if its worker outlives the shutdown cancel grace, the pool is deliberately leaked (with a log line) rather than destroyed under a live worker — a leak at process exit, never a use-after-free.

5 The exemplar: `gpt_scope`

`gpt_scope` trains a small character transformer and shows a per-head attention heatmap. It opts in lazily: on the first CUDA probe, if the host grants `IMPORT_ALLOC` and torch reports CUDA, it constructs one `ExportablePool`; on any failure the pool stays null and the applet is byte-identical to its pre-v1.2 self.

Two realities of the existing code shaped the integration. First, the tensor actually uploaded is not the raw attention snapshot but a block-upscaled derivative built at the upload site — so in pool mode the *worker* materializes the per-head upscaled tensors inside `pool.use()` (the worker is the only thread allocating CUDA memory during that scope, which keeps foreign allocations out of the pool). Second, a mapped texture’s value range is fixed at creation, and per-head ranges differ — which is precisely why the old code re-created the texture on every selection change. Pool mode instead pre-normalizes each head by its own maximum into a fixed `[0, 1]` mapping, so one texture is created once and every subsequent probe/selection change is an *update* — the operation the imported path accelerates. The heatmap status line says “zero-copy (imported pool)” only while the displayed texture’s latest content actually arrived

through `update_texture_from_alloc` (the platform reserves “zero-copy” wording for paths that literally are).

6 Verification

All results below were produced on the target machine: Windows 11, NVIDIA RTX 500 Ada (laptop), driver 596.47, Vulkan renderer, torch/libtorch 2.5.1 CUDA 12.1.

6.1 Byte-exact test rows

Five hardware-gated rows were added to the gfx suite, each driving a real `cuMemCreate` block through the public ABI (the host gained a separate *optional* `VmmApi` driver table so tests can build shareable allocations; the core driver table’s all-or-nothing rule is untouched). “Byte-exact” is literal: `==` on the RGBA8 readback against the CPU reference, no tolerances.

Row	Exercises	Expectation
f32 at offsets 0 and 512	descriptor-offset compute	" compute-imported ", byte-exact at 0 and 512
misaligned offset 4	alignment gate	false ; pixels and telemetry untouched
u8 RGBA at offset 512	blit bufferOffset	" blit-imported ", byte-exact; 3-channel false
release + reuse	id lifecycle	update-after-release false ; 50× import/release, ids strictly increasing
out-of-bounds window	both bounds gates	false for past-end and straddling windows

Table 1: The five imported-allocation rows. Full suite after integration: 22 cases, 1030 assertions, green; the pre-existing CUDA rows (device compute path, `alloc_shared`, burst pipelining) prove the refactoring left the donor paths byte-identical.

6.2 Live application

A live `gpt_scope` session on the Vulkan renderer: training on CUDA, with the renderer logging “*import path OK — zero-copy (imported allocation in place)*” on the first imported update, status line “*zero-copy (imported pool)*” (screenshot-verified at training step 344), head-selection switching through the update path, cancel and relaunch mid-run clean, and a mid-run applet unload driving the pool destructor with live imports — no crashes, no validation errors, no frame-watchdog stalls. Under `CALIPER_RENDERER=gl` the same applet trains and renders identically via CPU staging (the log shows the expected foreign-device rejection), demonstrating the fallback intact.

6.3 Findings worth recording

Three defects were found and fixed *only because* the code first ran on real hardware, and one was a specification bug:

- **SECURITY_ATTRIBUTES required.** On this driver, a WIN32-shareable `cuMemCreate` carrying a null `win32HandleMetaData` returns `CUDA_ERROR_INVALID_VALUE` (surfaced, confusingly, as a torch “OOM”). An exportable NT handle needs security attributes — the CUDA `memMapIPCDrv` sample’s precedent.
- **Cold-start ordering.** `beginAllocateToPool` before torch’s lazy CUDA initialization terminates the process; the pool constructor now forces `lazyInitCUDA()`.
- **Spec geometry bug.** The handoff’s first test row specified a 17×9 f32 grid at offset 0 and a second grid at offset 512 — but $17 \times 9 \times 4 = 612 > 512$: the regions overlap, and the

renderer honestly rendered the clobbered bytes while the reference assumed pristine data. The row now writes, updates, and verifies each region in sequence. The failure mode (first divergent byte exactly at output byte 512) is a small advertisement for byte-exact assertions.

- **Review-found lifetime bugs.** A final whole-branch review surfaced the release-ordering and threading issues summarized in the Lifetime section; all were fixed and re-verified before this report.

7 Limits and non-claims

- The floor is removed only for memory born in the exportable pool; default-allocator and foreign-allocator tensors keep the one-copy interop path. This is inherent, not incidental: exportability is an allocation-time property.
- Hardware verification covers Windows + NVIDIA + Vulkan. The POSIX file-descriptor handle path is stubbed and inert; Linux import is a port task, not a claim.
- In pool mode the exemplar pre-normalizes attention values in f32 before the shader maps them; against the old path this can differ by at most one 8-bit quantization step. The cap-absent path is byte-identical to the pre-v1.2 applet.
- The imported bytes must stay stable between an update and the texture’s next update; the ABI documents this, and at the exemplar’s probe cadence the window is negligible, but a pathological producer could violate it.

8 Conclusion

The interesting move was recognizing that “arbitrary tensors cost one copy” was never a statement about tensors — it was a statement about where their memory was born. Torch’s pluggable-allocator machinery lets an applet change that birthplace for exactly the tensors it wants to display, and one imported block then serves unlimited zero-copy texture updates at byte offsets. Everything else in the design — the additive ABI, the two-layer bounds checks, the fail-closed ladder, the wording discipline — exists to make that one move safe to ship on a frozen platform. The floor still exists; it just moved to where it belongs: on memory that genuinely cannot be shared.